

2주차 - 컴퓨터의 논리와 구조

디지털 혁명

디지털 기술 -> 사회적, 경제적 면에서 **획기적인 변화** 유발

사회 전반에 걸친 패러다임을 근본적으로 변화시키고 있음

IT 산업의 팽창

IT 기술의 여러 산업에의 적용

전자상거래의 확산

정보 공유의 확대

아날로그에서 디지털로, 오프라인에서 온라인으로

디지털 기술의 고도화로 전자 및 미디어 분야가 디지털 산업으로 변화

디지털 세상의 4가지 핵심 아이디어

1. 보편화된 정보의 디지털 표현
2. 보편화된 디지털 처리 장치 컴퓨터, 단말...
3. 보편화된 디지털 네트워크
4. 어마어마한 양의 디지털 데이터

디지털 혁명

정보처리(Information Processing)

데이터를 가공하고 처리하여 정보를 생산하는 과정으로 컴퓨터의 발전과 함께 활성화

정보화

‘고도의 정보통신 기술 혁신’을 배경으로 정보를 사회 전 분야에 널리 확산시키는 것

정보화의 핵심 : 컴퓨터, 방송, 통신 기술의 발전 + 네트워크 중심의 쌍방향통신으로의 변혁

디지털 전환(Digital Transformation)

- * 기업에서 정보통신기술(ICT)을 플랫폼으로 구축·활용하여 기존 전통적인 운영 방식과 서비스 등을 혁신하는 것
 - * 디지털과 물리적인 요소를 통합하여 비즈니스 모델을 변화시키고, 산업에 새로운 방향을 정립하는 전략 by IBM
 - * 전산화(digitization, 아날로그>디지털) + 디지털화(digitalization, 정보통신기술 활용)
- 사례 : GE의 산업 인터넷용 소프트웨어 플랫폼 ‘프레딕스’, 스타벅스 ‘사이렌오더’

컴퓨터의 등장

초기 계산기의 발달 과정

기원전 26세기	중국	주판
1617	네이피어	봉
1642	파스칼	토피바퀴 계산기
1732-1834	재퀴드	방직기
1823	배비지의	차분 엔진
1834	배비지	분석 엔진 +기억, 연산, 입출력 장치를 갖춘
1889	홀러리스	천공 카드
1944	에이킨	마크-1 초기 컴퓨터, 세계 최초의 전기 기계식 자동 계산기

1세대 컴퓨터

1951-1958

진공관 : 데이터 저장, 처리에 사용

자기드럼 : 주기억장치

천공카드 : 입력, 출력, 보조기억장치

프로그램은 **기계어**를 사용하여 작성 -> 프로그래밍하기 어려움

폰 노이만 - 범용 컴퓨터의 등장

HW, SW가 나누어지고 있다.

2세대 컴퓨터

1958-1963

진공관 > 트랜지스터로 바뀜

크기 작아짐, 열 발생 적어짐

가격 저렴해짐. > 컴퓨터 기술 진보에 결정적 계기

주기억장치(메인 메모리, 램)의 발달, 운영체제 도입

OS 도입

3세대 컴퓨터

1964-1970

반도체의 등장

중앙처리장치 소형화, 메모리 커짐

컴퓨터 하드웨어 능력 향상, 다양한 SW를 실행할 수 있게 됨

C, Pascal, Basic 등 다양한 프로그래밍 언어

SW 체계 확립(관리, 처리, 사용자 프로그램 등), OS 등장

4세대 컴퓨터

1971-현재

고밀도 집적회로(LSI; 손톱크기에 수만개 소자 집적), 초고밀도 집적회로(VLSI)

초대형 컴퓨터는 연산속도가 피코초(pico)

개인용 컴퓨터의 급속한 보급. 사무자동화(OA), 공장자동화(FA), 가정자동화(OA) 붐

컴퓨터 통신망 및 인터넷 발달

5세대 컴퓨터

현재

미래지향의 앞서는 컴퓨터

인공지능 기반. 양자, 나노, 분자 컴퓨팅 자연어로 대화하고 스스로 생각하고 학습하는 능력 목표
주요기술

* 초전도기술, 초고속장치

* **병렬 컴퓨팅** : 여러 개의 컴퓨팅 장치가 연결되어 하나의 작업을 처리

* 논리적 추론 지원 연산 가능

* 인공 지능 : 인식, 판단과 검색, 예측과 추론, 학습

컴퓨터 시스템의 구성

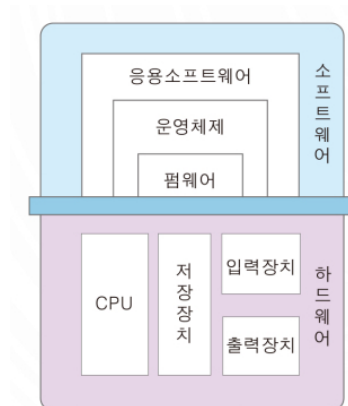
컴퓨터 시스템 : 하드웨어 + 소프트웨어

하드웨어 : 전자회로 + 그 밖의 물리적인 장치

소프트웨어 : 하드웨어를 활용할 수 있는 프로그램이나, 관련 기술

컴퓨터 하드웨어의 물리적 구조

버스 : 다양한 HW끼리 연결되어 있음. 통로



보조기억장치(Secondary storage)

주기기억장치보다 많은 정보 저장. 휘발성을 띄지 않음(전력이 꺼져도 정보 유지)

	HDD	SSD
특징	자기장의 원리로 정보 저장	플래쉬 메모리 (Flash memory) 개별 소자에 전하를 유지하는 방식
강점	가격이 저렴 장기 보존이 가능	읽기/쓰기 속도 느림 충격에 강함 전력소비가 적음 소음과 발열 적음
약점	읽기/쓰기 속도 느림 충격에 약함 전력 소비가 큼 소음과 발열이 있음	가격이 비쌈 자연적인 방전으로 데이터 손실 위험

>>> SSD 속도 빠름.. 오타!

추상화(abstraction) : 메모리의 물리적 구조는 다양하지만, 논리적 역할과 구조는 동일함. 실제 사용자들이 HDD와 SSD의 차이를 몰라도 보조기억장치를 사용하는 데는 문제가 없다. 그것이 추상화!

입력장치

데이터 -> 전기적 신호 장치로 변환시켜주는 장치
키보드, 마우스, 터치스크린, 센서, 카메라, 마이크 등

출력장치

CPU가 처리한 결과를 출력하는 장치
모니터, 스피커, 프린터, 플로터

네트워크 장치 : 와이파이, 블루투스

USB : Universal Serial Bus. 보조기억장치 아님! Bus이다. 기판의 확장

프로세서 칩

HW 장치의 크기는 줄고, 용량과 성능은 빨라짐
기판 : 부품 수 줄고 작아짐. 회로들이 부품 내부로 들어가 단순화. 배선을 미세하게 배치, 핀은 조밀하게 배치
IC(집적회로. Intergrated Circuits) 칩
최소단위 계산을 하는 **논리 게이트의 집합체**
실리콘으로 만드는 반도체 소자

무어의 법칙

반도체 칩에 집적할 수 있는 트랜지스터의 숫자가 적어도 매 1년(->18개월->2년)마다 두 배씩 증가한다.
그러나, 더 이상 무어의 법칙이 이어지지 않는다.
칩이 너무 빨라져 **열을 너무 많이** 발생
칩 자체의 속도 향상보다 **칩을 여러 개** 사용해서 성능을 향상
단, **메모리 용량**은 기하급수적으로 늘어나는 중

3주차 - 아날로그 정보를 디지털로 바꾸기

연속(아날로그)과 불연속(디지털)

정보를 재료로 무언가를 **진행(처리)**하는 것이 컴퓨터의 역량

컴퓨터가 처리하는 재료, 정보의 표현에 대한 아이디어

* 컴퓨터는 **디지털** 처리 장치; 디지털은 곧 숫자

* 정보를 **비트**(이진 숫자. 0과 1. binary number)로 표현

* 비트가 모이면 생각보다 **크고 많은 정보를 표현**할 수 있다. ;재료, 프로그램의 명령어

아날로그, 연속		디지털, 불연속
핸들		방향지시등
아날로그 시계		디지털 시계
아날로그 속도계		디지털 속도계

디지털의 장점

다루기 쉽다.

암호화할 수 있다.

다른 데이터와 **병합**되고, 그대로 **복사**되고, 인터넷으로 쉽게 어디로든 **옮겨**지고, 다양한 장치에 **저장**될 수 있고, 쉽게 **확장**될 수 있다.

이미지의 디지털화

실세계 장면은 **아날로그** 형태로 존재

디지털 이미지는 **픽셀**로 구성 - $1000 \times 1000 = 1,000,000$ 픽셀

RGB 컬러모델 : 각 픽셀을 표현하기 위해 (R,G,B) 각 1바이트 차지

ex) $1,000,000 \text{ pixel} \times 3 \text{ 바이트} = 3 \text{ 백만 바이트}$

>> 대표적 이미지 **압축** : JPEG

앨리어싱(aliasing) 알리어싱 : 아날로그 -> 디지털로 바꾸면서 생기는 계단 현상. **안티 알리어싱**

이미지의 데이터 처리

이미지의 색표현

이미지의 합성 : 이미지 2개를 투명도 조정하기

Morphing 모핑 기법 : 얼굴 합성 등..

#사운드의 표현

사운드 : 귀로 들을 수 있는 모든 청각정보 : 음악, 음성, 음향효과의 세 분야

사운드의 요소 : 주파수, 진폭, 음색

가청 주파수 : 20Hz~20KHz

아날로그 파형을 디지털 형태로 변환하기 위하여 표본(Sample)을 취하는 것을 의미. **샘플링**

사운드의 표본화율(**Sampling Rate**)

표본화율이 낮으면, 차지하는 메모리 용량은 작아지지만, 원음을 잘 반영하지 못함

입력장치 -> A to D converter -> 메모리/디스크 -> D to A converter -> 출력장치

#사운드의 처리

진폭의 크기 - 소리의 크기

주파수의 주기 - 음색

한꺼번에 들리는 소리는 두 데이터의 합

#BCD, EBCDIC

문자 데이터의 표현

한 글자를 컴퓨터로 표현하거나 입력하기 위해 그 글자의 바이트 값을 정함.

키보드로 입력하면, 이를 대응되는 바이트 값으로 저장하고, 이 대응되는 글자로 표현해준다.

아스키(ASCII; American Standard Code For Information Interchange) 코드 : 바이트-글자.

8비트로 구성. 첫 비트는 sign비트, 나머지 7비트로 128가지 표현

A - 65 a - 97 대문자+32=소문자

한글 데이터의 표현

완성형(KSC 5601) : 한글 2350자, 특수문자 130자, 숫자 20자, 한글날자 40자, 기본한자 4888자.

모든 글자가 이미 정해져 있음. 글자체 모양 좋음

조합형 : 현대 한글 음절 11172개 모두 표현 가능. 초, 중, 종성 별도로 처리.

유니코드(UNICODE) : 전 세계 언어를 하나의 코드 체계 안으로 통합하려는 컴퓨터 업체들의 협의로 만들어짐.

2바이트로 각 국가의 언어를 표시할 수 있음.

유니코드를 지원하면, 언어에 대한 별도 처리 없이 언어를 볼 수 있음. (호환성)

0과 1의 세계

이진법(Binary) : 0과 1의 두 가지 상태

On/Off True/False 물이 차 있음/물이 비어 있음. 5V/0V

비트 : 0과 1과 같은 가장 작은 정보를 나타내는 단위

바이트 : 8비트. 영어 한 글자 표현

워드 : 한 번에 처리하는 데이터 크기. 컴퓨터마다 다름. 32비트 컴퓨터에서는 32비트

1KB= 2^{10} Byte 1MB = 2^{20} Byte 1GB, 1TB, 1PB

8진수->10진수 2진수->10진수 10진수->2진수
2, 8, 16진수 상호 변환

멀티미디어 정보의 해석

멀티미디어 데이터 : 숫자, 문자, 사운드, 이미지, 그래픽, 동영상 등... 동일한 바이너리 데이터를 다르게 해석

* 데이터 용량이 매우 크다 > 데이터의 압축

* 용도와 환경에 따라 다양한 형태로 표현

디지털 데이터의 장점

아날로그 형태는 그 특성이 달라 저장, 처리, 전송 방식이 모두 다르지만, 디지털 데이터는 동일하게 다룸

->

디지털 형태는 **호환성** 높고, **저장,처리** 쉽고, **데이터 전송 오류나 손실의 문제점 최소화**함.

4주차 - 프로세서

프로세서(CPU, 중앙처리장치)

제어장치, 연산 장치, 레지스터 등으로 구성됨
개인용 컴퓨터에서는 마이크로프로세서라고도 부름

프로세서의 작동 범위

수의 계산 - 산술 연산 + - * / %

메모리에서 데이터 가져오기, 저장하기 (load, store)

입력과 출력

결정(decision) - 비교연산, 논리연산

=> 사람의 개입이 없어도 현재 처리중인 데이터를 기반으로 다음 작업을 결정할 수 있으므로 스스로 전체 시스템을 작동할 수 있게 한다.

폰노이만의 프로세서 : 기계는 완전히 자동이 되어야 한다. 즉, 계산이 시작된 후에는 운영자에게 독립적이어야 한다.

모형 컴퓨터(Toy Computer)

컴퓨터의 속성을 갖추고 있는 컴퓨터. 1960년대 말 미니컴퓨터 수준.

명령어와 데이터를 저장하기 위한 메모리

한 개의 수를 담을 만한 용량의 저장 용역인 **누산기(accumulator)**

GET	PRINT	STORE M	LOAD M	ADD M	STOP
-----	-------	---------	--------	-------	------

명령어(instruction) 동작의 사이클

fetch(인출/호출) -> decode(해석) -> execute(실행) : 이게 한 명령어를 처리하는 사이클

모형 컴퓨터 시뮬레이터

get	get a number from keyboard into accumulator
print	print contents of accumulator
load <i>Va</i> /	load accumulator with <i>Va</i> / (<i>Va</i> / unchanged)
store M	store contents of accumulator into memory location called M (accumulator unchanged)
add <i>Va</i> /	add <i>Va</i> / to contents of accumulator (<i>Va</i> / unchanged)
sub <i>Va</i> /	subtract <i>Va</i> / from contents of accumulator (<i>Va</i> / unchanged)
goto L	go to instruction labeled L
ifpos L	go to instruction labeled L if accumulator is >= zero
ifzero L	go to instruction labeled L if accumulator is zero
stop	stop running
M Num	before program runs, set this memory location (called M) to numeric value Num

Acc에 값을 입력받아 그 값을 출력해라

GET
PRINT
STOP

두 수의 값을 입력받아 그들의 합을 출력해라

GET
STORE FirstNum
GET
ADD FirstNum
PRINT
STOP

FirstNum 0 <<< 변수처럼 선언해줘야 작동

GOTO문을 사용한 조건문

```
Top :   GET
        PRINT
        IFZERO Bot <- Acc가 0이면 Bot으로
        PRINT
        GOTO Top
Bot:     STOP
```

GOTO문을 사용한 조건문

```
Top :   GET
        PRINT
        IFZERO Bot // Acc가 0이면 Bot으로
        ADD Sum // Sum의 값을 Acc에 더하기
        STORE Sum //Sum의 값을 다시 Acc에 저장
        PRINT
        GOTO Top
Bot:     LOAD Sum //Sum을 Acc에 load
        PRINT //Print는 Acc의 값을 출력
        STOP
Sum: 0 // 변수 선언
```

메모리의 내부 표현

명령어 코드 : GET=1, PRINT=2, STORE=3, LOAD=4, ADD=5, STOP=6, GOTO=7, IFZERO=8
GET, PRINT, STOP < 메모리 1칸만 차지
ADD sum, STORE sum < 메모리 2칸 차지

프로세서는 빠른 것이 좋을까?

프로세서가 하는 일 : 인출/해석/실행 과정을 반복 수행

컴퓨터 아키텍처(Computer Architecture)

프로세서 설계 + 프로세서와 다른 하드웨어 요소 간의 **연결 방식 설계**

명령어 집합(Instruction Set) : 프로세서가 할 수 있는 일들의 집합. Machine dependent

폭넓고 다양한 **많은 수의 명령어** vs 작성하고 쉽고 더 빨리 실행되는 **적은 수의 명령어**

=> 기능성, 속도, 복잡도, 전력 소모 정도, 프로그램 가능성(Programmability) 간 트레이드오프

레지스터와 캐시

: 프로세서의 가까이 **빠른 메모리**를 두고 중간 결과값이나 자주 사용하는 명령어나 데이터를 **미리 담아두는 역할**

파이프라이닝

: 데이터 처리에서 여러 단계가 서로 동시에, 병렬적으로 실행되는 것

병렬 컴퓨팅(멀티코어 프로세서)

: 프로세서 여러 개를 사용해서 한 작업을 여러 프로세서에서 처리하는 것. (한 컴퓨터 안에서 실행)

∴ 컴퓨팅 기기의 특성마다 프로세서가 다름

데스크탑 / 휴대용 컴퓨터 / IoT 기기

인텔, AMD, ARM

>> 프로세서간에 속도를 비교하는 것은 어렵고, 그다지 의미가 없음.

>> 어떤 프로세서가 다른 것보다 '빠르다'라는 것은 조심스럽게 받아들여야 함. 요구사항에 따라 차이 있음.

캐시(Cache)

용량이 작고 속도가 빠른 메모리

주메모리에 매번 접근하는 것을 피하기 위해 최근에 사용된 정보를 저장

곧 다시 쓸 가능성이 있거나 근처에 있는(인접함) 뭔가를 사용한 가능성이 있을 때 활용할 수 있는 개념

if, 계속 순차적으로 작동하는 프로그램이라면, cache에 들렀다가 메모리로 가야해서 비효율적일 수도 있음!

슈퍼컴퓨터부터 사물인터넷까지

컴퓨터의 종류는 다르지만 **속성(하는 일, 논리적 구조)**은 같다.

컴퓨터는 모두 논리적 구조에 대해 공통적 핵심 속성을 가짐. 비슷한 아키텍처이나, 가격, 소모, 전력, 크기, 속도 등에서 트레이드 오프를 보임.

GPU : 화면, 이미지 데이터 처리에 특화된 프로세서

분산 컴퓨팅 : 네트워크로 연결해서 동시/분산 처리 가능

>>전반적인 아키텍처는 크게 바뀌지 않았으나, 물리적 구조는 빠르게 진화 중

5주차 - 알고리즘

범용 기계(General Purpose)

: 컴퓨터는 어떤 계산이라도 수행할 수 있는 범용 기계이다.

컴퓨터는 “**극도로 상세하게**” 할 일을 알려줘야 함.

시스템의 물리적 작동 방식을 바꾸는데 소프트웨어를 사용하는 일은 **항상 간단하지 않다**.

(특히 Critical System 크리티컬 시스템; 보잉 비행기 MACS, 자율주행차, 의료 영상 시스템, 군용 시스템에)
안전해야 하고, 보안조치가 되어있어야 하고, 고도의 검증이 되어야하는 시스템. 생명과 직결. 이런 시스템은 높은 수준의 검증이 필요함.

소프트웨어 테스트 : SW를 검증하는 것

소프트웨어

: 컴퓨터가 동작하게 하는 프로그램(레시피, 매뉴얼)

레시피(재료, 요리, 순서, 음식) vs 소프트웨어(데이터, 작업, 동작)

알고리즘

: 컴퓨터 과학에서의 작업 절차

: 결과를 정확하게 계산되도록 보장된 **일련의 단계**. 각 단계는 **연산**으로 표현

- * 알고리즘을 이루는 모든 구성 요소의 의미에 **모호함이 없어야** 함.
- * **입력** 데이터가 어떤 유형인지 제공해야 한다.
- * **모든 가능한 상황**을 다루어야 한다.
- * **다음에** 무엇을 해야 할지 모르는 상황이 발생하면 안 된다.
- * 알고리즘은 결국 **멈춰야** 한다.

>>알고리즘은 **상세하고 정확하게, 일련의 연산**을 명시해야 한다.

>>알고리즘을 **효율적으로 설계, 분석, 구현**해야 한다.

알고리즘의 효율성 : 처리 데이터 **양에 따라** 계산에 **소요되는 시간**을 표현하는 방법

선형 알고리즘

자료 구조 : 계산 과정에서 필요한 정보를 표현하는 방법.

탐색 알고리즘

- * 수많은 데이터 중 어떤 조건을 만족시키는 데이터를 찾아내는 과정
- ex) 웹 정보검색, DB의 SQL 질의어 등
- * **순차 탐색(Sequential Search)** : 앞에서부터 찾아보기

선형 시간(Linear-time) : 계산 시간이 데이터의 양에 정비례하거나 선형적으로 필요할 때 -순차 탐색

이진 검색 알고리즘

- : 분할 정복(divide and conquer)
- : 검색할 리스트가 **미리 정렬**되어 있어야 함.

$\log_2 N$: 문제의 범위가 절반씩 줄어들고 있으므로 입력 데이터 N 에 따른 알고리즘의 계산 시간.

정렬 알고리즘

데이터의 모임을 어떤 조건에 따라 **오름차순** 또는 **내림차순**으로 정렬

선택 정렬 알고리즘 :

최소 숫자를 선택할 때마다 살펴볼 데이터의 양이 한 항목 줄어들고 목표를 이루기 전까지 같은 작업을 반복
 $N^2 + N$

퀵 정렬 알고리즘 : 살펴볼 데이터를 반으로 줄여가며 목표를 이루기 전까지 같은 작업을 반복한다. 어려움...
 $N * \log_2 N$

알고리즘의 성능

시간복잡도 : CPU 평균 시간과 최악 시간

공간복잡도 : 메모리 공간

빅-O 표기법 : 데이터 크기 N 이 증가함에 따라 알고리즘 성능의 변화도

빅오 표기법은 알고리즘의 효율성을 표기해주는 표기법이다. 알고리즘 최악의 실행 시간을 표기함.

여행하는 외판원 문제

최적화 문제에서 해를 찾을 때 항상 '욕심 내어' 최소값 또는 최대값을 찾아가는 알고리즘

어떤 특정 시점에서 최선의 선택을 하는 알고리즘

휴리스틱(경험기반).. best는 아니지만 효율적인 방법 $2^n \rightarrow n^2$ 지수알고리즘을 다항식 알고리즘으로 바꾸어 효율적으로 함.

aka 탐욕 알고리즘(Greedy Algorithm). ex)최소 스패닝 트리

지수 알고리즘

모든 가능한 경우를 하나씩 시도해 봐야만 하는 상황에서 발생

암호기법에서 사용되는 알고리즘

$O(2^N)$

NP(Nondeterministic polynomial) 문제 ;어려운 문제 vs P(Polynomial) 문제; 쉬운 문제

특정한 기준을 줘서

모든 케이스를 다 찾아야 하는 문제를 요령(휴리스틱)을 써 빨리 찾으려고 하는 노력 : 알고리즘

N 이 작으면... 그냥 구현하기 쉬운 거 쓰는 게 좋을수도...

N 이 크면... 효율적인거 써야 할 수도...

6주차 - 알고리즘과 프로그래밍

#알고리즘과 프로그래밍(프로그램)

알고리즘은 정확하고 명료한 레시피.

추상적이고 이상적인 절차를 기술한 것을 구현에 필요한 세부 사항과 현실적인 고려사항 무시 의미가 완전히 알려져 있고 구체적으로 명시된 기본 연산으로 각 단계를 설명하고 모든 가능한 상황을 다루는 가이드라인

프로그램은 구체적인 작업 명세서

실제 '컴퓨터'가 과제를 완료하기 위해 수행해야 하는 모든 단계를 구체적으로 서술

하나 이상의 알고리즘이 컴퓨터가 직접 처리할 수 있는 형태로 표현된 것

알고리즘이 하지 않은 실질적인 문제에 대한 고려 **프로그래밍할 때 고려해야 할 사항.**

ex) 불충분한 메모리, 프로세서 성능, 잘못된 입력 데이터, 하드웨어 결함, 네트워크 연결 불량 등

>>같은 알고리즘이어도 하드웨어(스마트폰, 컴퓨터...)에 따라 실제 구현(프로그램)은 다를 수 있다.

#최초의 전자식 컴퓨터에서의 프로그래밍은 진입장벽이 높았음.

명령어와 데이터를 이진수로 변환 -> 펀치카드(천공)로 프린팅 -> 기계판독하여 메모리로 저장

이 과정을 수동으로 하는 것은 예민하고 어려운 작업

오류 발견되면..? 기능을 추가/변경 해야 한다면..?

#어셈블러 의 등장

프로그램 속 명령어를 나타내는 키워드(SUM, ADD 등)을 코드(숫자) 대신 매핑해주는 프로그램

어셈블러가 명령어와 데이터 값이 메모리의 어느 위치에 있을지 지정하거나 파악해 줌

어셈블리 언어, 어셈블리 언어 프로그래밍

#어셈블리 언어

프로세서의 명령어와 일대일로 연결! -> 기계 종속적임. 기계에 따라 다름

특정 장치에서 지원하는 명령어의 이진수 전환 방식과 메모리 할당 방식에 맞추어 동작

#컴퓨터가 프로그래머를 대신해서 더 많은 일을 수행하려는 방향으로

기계어 -> 어셈블러 -> 고수준 프로그래밍 언어 -> 노코드 프로그래밍

#고수준 프로그래밍 언어(high-level PL) 등장

특정 프로세서 아키텍처에 독립적인 프로그래밍 언어(하드웨어 의존도 낮춤!)

번역기 프로그램(컴파일러, 인터프리터)이 고수준 언어로 작성된 코드를 대상 프로세서의 어셈블리 언어로 변환

컴파일러 : 고수준 -> 어셈블리어

한번에 번역본 만들어 놓기

인터프리터 : 한줄한줄 해석

#고수준 언어의 강점

배우고 사용하기 더 쉬움

특정 프로세서의 명령어 레퍼토리를 알아야 할 필요가 없음

특정 아키텍처에 종속되지 않음. 프로그램을 만들면 여러 아키텍처에서 실행될 수 있음.

FORTRAN(포트란) : 과학, 공학 분야에서 계산을 표현

COBOL(코볼) : 사무 데이터 처리 목적

BASIC(베이직) : 범용앱 - 간단하면서도 컴퓨팅 자원을 적게 필요. 개인용 컴퓨터에서 사용할 수 있는 첫 번째 고수준 언어

#C언어 - 시스템 프로그래밍

1973년 데니스 리치(Dennis Ritchie)가 개발
운영체제, 프로그래밍 도구 : 어셈블러, 컴파일러, 텍스트에디터

#C++ - 규모가 큰 프로그램의 복잡성 관리용

1980년대 목표로 비야네 스트롭스트룹에 의해 개발
객체지향 프로그래밍(OOP) - 객체와 클래스 개념 도입...

#JAVA 자바 - 효율적인 프로그램보다, 빠르고 편리한 프로그래밍

가전제품, 전자기기와 같은 임베디드 시스템에 장착을 목표로 개발(1990년대 초)
웹 페이지를 생성하는 용도로 웹 서버에 사용되고 있음
안드로이드 앱을 개발하는 주요 언어

#JavaScript 자바스크립트 효율적인 프로그램보다 빠르고 편리한 프로그래밍

웹페이지의 동적인 효과를 구현하기 위해 브라우저 내부에 사용할 목적으로 설계
언어가 단순하고 컴파일러가 모든 브라우저에 내장되어 있어 작업하기 용이

#Python 배우기 쉽고 읽기 쉬운 프로그래밍

생각할 수 있는 거의 모든 프로그래밍 과제에 필요한 라이브러리를 풍부하게 제공
최근에 가장 선호되는 프로그래밍 언어

#앞으로 어떤 언어가 등장할까?

효율보다는 접근하기 쉬워야... vs 그래도 빠르고 효율적이어야....

#어떤 언어를 사용하건 더 중요한 건 내용

언어는 우리가 생각하는 방식을 형성하고, 생각할 수 있는 범위를 결정한다.

#노코드 개발

개발자의 업무를 지원하는 '로우코드'와 코딩이 필요하지 않은 '노코드'
노코드 예시 : 1. Microsoft Power Platform 2. AppGyver 3. OutSystems

#구글같은 서비스는 어떻게 개발할까? 개발 프로세스

1. 문제 해석 및 요구사항 분석 : 무엇을 해야 할지 파악 -> 명세서 나옴...
2. 설계 : 넓은 명세서부터 시작해서 점차 작은 부분으로 적절히 나눔 -> 데이터 구조 등 정하기...
3. 개발/통합배포/테스트 : 각 부분을 작업하면서 전체적으로 일관되어 있는지, 문제 없는지 테스트
 폭포수 방식 : 개발 시 각 단계별 완성도를 높여서 중간에 변동이 생기지 않게 하는 방식
 애자일 방식 : 그때그때 민첩하게 유동적으로 바꿈.

#라이브러리, 인터페이스, 개발키트

: 기존의 노하우를 빌려와서 사용

라이브러리 : 이미 잘 작성된 동작문치들의 집합

다른 사람이 만들어 둔 것을 사용하거나, 내가 라이브러리를 만들어 두고 사용하거나...

API(애플리케이션 프로그래밍 인터페이스)

: 라이브러리로 배포하는 함수의 이름과 입출력 데이터의 구조를 상세하고 정확하게 설명하는 명세서
프로그래머가 서비스를 요청하기 위해 무엇을 해야 하고 결과적으로 무엇이 계산될지 정의
공개 API 어떤 데이터를 주고받아야하는지 미리 약속이 된 것. 원격으로 접속도 가능...
다른 프로그램들이 서로 소통하고 데이터를 주고받을 수 있도록 연결해주는 다리

개발 키트(SDK) : 라이브러리 + API + API 사용법 + 개발 플랫폼 + Documentation(문서)

#버그 : 기계, 계획 등에 생긴 흠이나 결함

발생할 수 있는 경우를 고려하지 못하고 처리하지 않았거나
조건을 평가하기 위한 논리 테스트나 산술 테스트를 잘못 작성했거나
잘못된 공식을 사용했거나
영역 밖의 메모리에 접근했거나
데이터 유형에 맞지 않는 잘못된 연산을 적용했거나
사용자 입력의 적합성을 검사하지 못했거나

>>프로그래밍은 완벽하기 어렵다

테스트를 얼마나 잘 하는가 → 디버깅의 기회 → 소프트웨어 결함 예방

#소프트웨어의 유지보수(Maintenance)

급변하는 환경에 얼마나 유연하게 적응하는가?
프로그래밍 환경(언어)이 바뀌기도 하고
프로그램을 탑재할 플랫폼이 바뀌기도 하고
개발자가 바뀌기도 하고
사용자의 요구사항 (SW 트렌드)이 바뀌기도 하고

>> 소프트웨어 아키텍처(건축가, 구조) : 대규모 프로젝트를 잘 할 수 있는가? 전체 과정을 이해하는가?

>>프로그래밍을 할 때 수정하거나 확장하기 쉽도록 구조화하고 명세서를 잘 남기는 것이 중요!!!

7주차 - 디지털 콘텐츠의 저작권

#지적재산권

발명이나 저작(책, 음악, 그림, 사진 등)과 같은 개인의 창작 활동에서 생겨난 다양한 종류의 무형 자산
소프트웨어는

무형이지만 가치가 높고

대량의 코드를 작성하고 유지보수하기 위해 지속적인 노력이 필요하고

무제한으로 복제 가능한데 비용은 들지 않으며, 손쉽게 변형 가능하고 눈에 보이지 않음

-> 소프트웨어의 법적 보호에 관한 제도는 아직 무르익지 않았음.

#영업비밀

소유자만 비밀을 보유하고, 기밀유지협약서 등 법적 구속력이 있는 계약에 의해서만 공개
간단하고 효과적, 비밀이 드러났을 때 대처 수단이 거의 없음

#상표

회사의 상품이나 서비스를 구별해 주는 단어나 문구, 이름, 로고, 특정한 색상을 의미

#저작권

문학, 미술, 음악, 영화와 같은 창작 표현물을 보호하기 위해

다른 이들이 복제하지 못하도록 창작물을 보호

창작자에게 제한된 기간 동안 작품을 활용하고 수익을 얻을 권리를 줌.

#디지털 저작권 관리(DRM)

유통의 전 과정에서 불법복제로부터 저작원자 권리를 보호하는 기술과 서비스.

사용자에게 주어진 권한에 따라 콘텐츠의 이용을 통제 : 보안기술과 비교해서 DRM이 더 포괄적인 개념!

두 가지 보호 방식

수동적 : 법적, 도덕적 자각심을 유도하여 스스로 불법행위 자제하는 효과
크리에이티브 커먼즈 라이선스(CCL), 디지털 워터마킹

능동적 : 암호화 기술로 사용 권한에 따른 디지털 콘텐츠의 접근제어

#CCL 크리에이티브 커먼즈 라이선스

자신의 저작물에 대하여 '이용방법 및 조건'을 콘텐츠에 표시.

UCC환경 등에서 콘텐츠를 공유할 수 있는 여건 제공

1. 저작권 표시 - 저작물의 원작품이나 복제물을 공표하거나 이용하려면 반드시 저작자의 성명을 표시해야 한다.
2. 비영리 - 저작물을 영리 목적으로 이용할 수 없다. 영리 목적으로 이용하려면 저작자와 별도 계약이 필요하다.
3. 변경 금지 - 저작물을 이용한 2차 저작물은 물론 저작물의 내용, 형식 등의 단순 변경도 금지한다.
4. 동일조건 변경허락 : 2차 저작물의 작성을 허용하되, 원 저작물과 동일한 내용의 라이선스를 적용해야 한다.

디지털 워터마크

인간이 쉽게 감지할 수 없는 표식을 문서, 그래픽, 오디오, 비디오 등의 멀티미디어 콘텐츠 내에 삽입하여 불법복제를 방지

원본의 출처 확인 가능, 전자상거래나 각종 증명서 발급에도 확인. 도장을 디지털 워터마크로 구현하기도 함.

디지털 핑거프린팅

저작자 정보가 아닌, 구매하는 사용자의 정보를 콘텐츠에 삽입.

핑거프린팅 : 만진 사람의 지문이 상품에 찍힌다.

>> 워터마킹과 유사하지만 불법 배포를 추적하여 방지한다는 차이가 있음.

#능동적 보호 방식의 DRM

- 원천적으로 못빼가게 보호

복제 방지 기술

저장매체/재생장치에 연동된 암호 키를 사용하여 암호화

다른 장치에서 사용 불가, 복제된 콘텐츠 사용 불가, 복제 자체도 불가

접근 제어 기술

정당한 사용권을 가진 사용자만 접근을 할 수 있도록 접근권한 제어

CAS(Conditional Access System)이 대표적. 유료 방송 채널을 보호하고 사용료를 징수

사용 제어 기술 : 협의의 DRM

사용자가 별도의 권한을 허락 받아 콘텐츠를 이용 - 지불한 콘텐츠에만 접근 가능하게

허가된 사용권한에 따라 디지털 콘텐츠의 사용을 통제 - 레벨별로

콘텐츠의 전체 유통과정에서 다양한 형태의 요금 부과가 가능

#소프트웨어 프로그램의 저작권

내가 작성한 프로그램은 내 소유이므로 내 허가 없이 사용할 수 없다.

어떤 프로그램의 작동 방식을 연구해서 나만의 버전을 만든다면? (범위가 문제....)

서식과 변수 이름만 바꾼다면

리버스 엔지니어링 : 소스코드를 역으로 분석.

해당 프로그램의 로직을 이해해서 새롭게 구현한다면 -

클린 룸 개발(clean room development) : 특정 소프트웨어를 참고하거나 복제하지 않고, 순수하게 독립적으로 자체적으로 새로운 소프트웨어를 개발하는 것

#특허 (Patent)

발명(의 내용)에 대한 법적 보호를 제공한다.

cf. **저작권**은 표현물을 보호하고 코드가 포함할 수 있는 **독창적인 아이디어를 보호하지 않음**

소프트웨어는 특허가 될 수 있는가?

문제해결 과정(알고리즘)의 독창성을 인정할 수 있는가?라는 질문과 연계될 수 있다.

발명이 나온 당시에 '해당 기술 분야에서 통상의 지식을 가진 자'에게 '자명하지 않아야' 한다.

소프트웨어 특허 괴물 (patent roll)

발명을 사용하려는 것이 아니라 허가 없이 특허를 침해 중인 다른 기업에 소송을 제기하기 위한 근거로 활용

#라이선스

제품을 사용할 권한을 승인하는 법적 권한

소프트웨어 **사용자**가 동의하는 라이선스 : EULA (end user license agreement)

소프트웨어 **개발자**가 동의하는 라이선스 : API 사용권한 (application programming interface)

인터페이스는 양쪽에서 무슨 일이 일어나는지 정의

: 서비스가 어떻게 구현되었는지에 대한 세부 사항이 아니라 함수가 프로그램에서 사용되는 방법 (프로토콜)에 관한 것

API는 특허를 받을 수 없어 **저작권의 보호**를 받도록 한다.

오라클의 API에 대한 저작권을 구글은 불법적으로 사용한 것인가, 공정사용(fair use)인가

#표준 - interoperability(상호운용성)

어떤 **기술적 산물**이 어떻게 만들어지고 어떻게 작동하도록 되어 있는지 **명확하고 상세하게** 기술한 것 (정규화된 표현)

개별 주체들이 의사소통하고 독자적으로 구현할 수 있을 만큼 **완전하고 정확**해야 함

정부 기관이나 컨소시엄 같은 준중립적인 **단체에 의해 개발**되고 유지

전원 플러그, 통합 리모컨, 스마트폰 충전기, USB ... IETF(Internet Engineering Task Force), ISO (

#표준의 중요성

표준은 **상호운용성**을 보장하고 **공개 경쟁**이 이루어지도록 하는 데 결정적인 요소.

독자적으로 만들어진 기술이 상호 협력할 수 있게 하고 여러 공급자가 경쟁할 공간을 제공

#오픈소스

소스코드와 오브젝트 코드(기계어)

오픈소스 : 연구와 개선 활동을 위해 다른 사람들도 **소스 코드를 자유롭게 사용할 수 있도록** 하는 대안을 일컫는 용어

자유 소프트웨어 재단(Free software foundation) 과 GNU(GNU is not Unix)

비독점적이고, 구속적인 소유권에 얽매이지 않는다 -> 영구히 '자유로운' 소프트웨어를 만들어 내자

GNU GPL(일반 공중 사용 라이선스)

GNU GPL(General Public License)은 자유 소프트웨어 재단(FSF, Free Software Foundation)에서 만든 라이선스 중 하나로, 소프트웨어의 자유로운 사용, 수정, 배포를 보장합니다. GNU 프로젝트의 일환으로 개발된 이 라이선스는 특히 소스 코드를 공개하고, 누구나 수정하거나 재배포할 수 있도록 하는 것이 핵심입니다.

GNU GPL의 주요 특징은 다음과 같습니다:

1. 자유로운 사용: 프로그램을 어떤 목적으로든 사용할 수 있습니다.
2. 소스 코드 공개: 프로그램을 배포할 때, 소스 코드도 함께 제공해야 합니다. 사용자가 프로그램을 수정할 수 있도록 보장하기 위함입니다.
3. 자유로운 수정 및 배포: 소프트웨어를 수정하거나 다른 사람에게 배포할 수 있습니다. 다만, 수정된 버전

도 원래의 GNU GPL과 동일한 라이선스 조건을 따라야 합니다(이것을 "copyleft"라고 합니다).

4. 동일한 라이선스 유지: GPL로 배포된 소프트웨어의 파생물은 반드시 동일한 GPL 라이선스를 유지해야 합니다.

이를 통해 소프트웨어가 특정 회사나 개인의 독점적인 제약 없이, 더 많은 사람들에게 공유되고 발전될 수 있도록 합니다.

사업 모델 : 소스코드를 개방하고 기술 지원, 교육, 품질 보증, 시스템 통합과 기타 서비스 이용에 요금을 쳐육하는 수익 모델

>> 오픈소스는 집단지성!

1바이트로 표현한 자연수 3개로 인식하는 경우
아스키코드 문자코드 인식문제
로우코드, 노코드 개념, 적용분야, 사례
병렬컴퓨팅, 분산컴퓨팅 개념, 공통점과차이점

단답) **크리티컬 시스템** 정의보고 쓰기

서술) 알고리즘을 프로그래밍하기 위해 **고려해야할 사항**, **큰 개발 프로젝트 진행 시 단계 서술**

단답) 3비트짜리 이진수 전환-1비트씩 자연수로 전환, 자연수 아스키코드 전환, RGB 색깔코드로 전환

서술) **GE사의 디지털전환 시도와 실패에 관한 분석과 교훈**

서술) 프로그래밍 라이선스 GPL 명시된 내용 설명

서술) 디지털 워터마킹과 핑거프린팅 정의와 사례 하나

꼬리문제 2개 : 디지털 전환 단답/GE사의 프레딕스 실패원인 분석 및 교훈 서술(과제랑 동일...)

3바이트 이진 코드 제시하고, 각각 자연수/ASCII/RGB로 해석할 시 무엇인지 단답(당연히 이진수->십진수 계산부터 가능해야)

CCL 제시하고 의미 설명. 그냥 강의자료만 봐도 패스

꼬리문제 2개 : 실제 프로그래밍 단계에서 고려해야 하는 요소 2가지 택하여 약술/큰 개발 프로젝트 진행 시 단계 서술

운영체제 목적 5개 중 3택 객관식

TOP: get
print
IFZERO bot
ADD sum
STORE sum
GOTO top
BOT: LOAD sum
STOP
SUM: 0

5
5
6
4
10
0

[stopped]

Accumulator: 30

Run

Clear output

Syntax reminder

get	get a number from keyboard into accumulator
print	print contents of accumulator
load <i>Va/</i>	load accumulator with <i>Va/</i> (<i>Va/</i> unchanged)
store <i>M</i>	store contents of accumulator into memory location <i>cal</i>
add <i>Va/</i>	add <i>Va/</i> to contents of accumulator (<i>Va/</i> unchanged)
sub <i>Va/</i>	subtract <i>Va/</i> from contents of accumulator (<i>Va/</i> unchanged)
goto <i>L</i>	go to instruction labeled <i>L</i>
ifpos <i>L</i>	go to instruction labeled <i>L</i> if accumulator is >= zero
ifzero <i>L</i>	go to instruction labeled <i>L</i> if accumulator is zero
stop	stop running
M Num	before program runs, set this memory location (called

1. 디지털 전환의 정의

디지털 전환(Digital Transformation)은 기업이나 조직의 모든 영역에 디지털 기술을 통합하는 과정으로, 프로세스, 제품, 운영, 고객 경험, 직원 경험 등이 디지털화됩니다. 넷플릭스, 우버 등의 사례처럼 기존 산업을 재편하는 방식으로 발전하고 있으며, 클라우드, 인공지능, 자동화 등 기술을 기반으로 혁신이 이루어집니다.

2. GE사의 디지털 전환 실패 분석

GE는 산업 인터넷용 플랫폼 Predix를 개발했으나, 글로벌 시장에 대한 이해 부족과 지나치게 광범위한 전환 시도로 실패했습니다. 특히 각 산업의 특성과 요구사항이 다름에도 이를 충분히 고려하지 않아 실패했으며, 교훈으로 는 소규모 전환과 시장에 대한 깊은 이해의 필요성이 강조됩니다.

3. 성공한 디지털 전환 사례

모바일 결제의 확산은 성공적인 디지털 전환 사례입니다. 네이버페이, 카카오페이 등 국내 서비스와 애플페이, 알리페이 등 국제적 서비스가 대표적이며, 모바일 결제는 전체 결제 비중에서 절반을 넘을 정도로 성장하였습니다.

1) 본인의 영문 이름을 아스키코드로 변환해 보시오.

2) 영문 이름을 저장하는 데 필요한 메모리는 몇 바이트인가?

아스키코드로 표현 가능한 문자(character)는 한 글자가 1바이트가 필요하므로 이름에 공백을 포함한다면 Shim Sungmin에는 12글자이므로 12바이트가 필요하다.

2. 이미지의 디지털 전환

카메라로 찍은 이미지를 아래와 같은 식으로 디지털 정보로 변환하여 파일로 저장하려고 한다.

주어진 이미지를 200x200의 해상도로 나눈다.

각 픽셀의 대표색을 뽑아낸다. 이 때, 이 이미지를 표현할 수 있는 색은 총 512개라고 하자.

이미지의 왼쪽 최상단의 픽셀부터 각 픽셀의 색 정보를 연속한 메모리에 저장한다.

1) 한 픽셀의 색을 표현하기 위해 필요한 최소한의 bit의 개수는?

이미지를 표현할 수 있는 색이 512개이므로 $512=2^9$, 따라서 최소 9bit가 필요하다.

2) 아래 이미지를 표현하기 위해 필요한 bit의 개수는?

이 이미지는 200x200 해상도이므로 $200 \times 200 = 40,000$ 개의 색상 정보를 표현해야 하고, 각 칸(pixel)은 1번에서 구한 대로 9비트가 필요하다. 따라서 총 필요한 비트 수는 $40,000 \times 9 = 360,000$ 비트가 필요하다.

Greedy Algorithm은 각 단계에서 최적의 선택을 하여 결과적으로 최적의 해결책을 찾는 알고리즘입니다. 이 알고리즘의 특징으로는 한 특정 시점에서 최선의 선택을 할 뿐, 그 이후에는 어떤 영향을 미칠지는 고려하지 않습니다. 따라서 Greedy Algorithm은 지역적 최적화를 통하여 전체적 최적화 목표로 하는 알고리즘입니다.

최근 응용 사례로는 허프만 코딩(Huffman Coding)이 있습니다. 저는 개인적으로 이 허프만 코딩에 대해서 전공수업인 정보보안론 수업에서 정보의 압축 알고리즘 내용을 배울 때 알게 되었는데, 이 허프만 코딩은 허프만 트리(Huffman Tree)라는 것을 기반으로 한 알고리즘입니다. 각 문자나 데이터 패턴의 빈도를 분석하여 빈도가 높은 문자는 짧은 코드로, 빈도가 낮은 문자는 긴 코드로 매핑하여 전체 데이터의 크기를 획기적으로 줄일 수 있는 압축 방법입니다. 허프만 코딩의 원리는 아래와 같습니다.

- 빈도 계산: 데이터에서 각 문자나 패턴의 빈도를 계산합니다.
- 노드 생성: 각 문자와 그 빈도를 포함한 노드를 생성합니다.
- 트리 구성: 노드들을 최소 빈도 노드부터 두 개씩 묶어 트리를 구성합니다. 이 과정을 반복하여 최종적으로 허프만 트리를 완성합니다.
- 코드 매핑: 허프만 트리를 따라 각 문자에 대한 코드를 생성합니다. 왼쪽으로 이동하면 0, 오른쪽으로 이동하면 1을 더합니다.

예를 들어 "aabacdb"라는 문자열을 허프만 코딩하기 위해서는 문자의 빈도를 구해야 합니다. a는 5번 b는 2번, c는 1번, d는 1번이 있는데, 이를 기반으로 허프만 트리를 구성하면 a = 0, b=10, c=110 d=111가 됩니다.

결과적으로는 문자열은 00110100011011로 변환됩니다. 앞서 언급했듯, 이 방법을 쓰면 데이터를 손실없이 획기적으로 압축할 수 있습니다.

1.노코드(No-Code)와 로우코드(Low Code)의 개념

- **노코드:** 코딩 없이 미리 만들어진 컴포넌트들을 조합해 드래그 앤 드롭 방식으로 기능을 구현하는 개발 방식. 코딩 경험이 없어도 웹사이트, 간단한 앱, 데이터베이스 등을 만들 수 있음.
- **로우코드:** 최소한의 코드만 사용하여 개발하는 방식으로, 시각적인 인터페이스와 코드 편집 기능을 결합. 노코드보다 복잡한 기능이나 시스템 연동에 적합함.

2. 주로 적용되는 분야

- **앱 개발:** 노코드/로우코드를 활용해 빠르게 아이디어를 프로토타입으로 전환 가능. 예: Glide.
- **데이터 시각화:** R과 SQL처럼 복잡한 구문을 배우지 않고도 시각화를 구현할 수 있는 노코드 도구가 많이 등장함. 예: 구글 데이터 스튜디오.

3. 대표적인 노코드/로우코드 플랫폼 사례

- **Glide:** 드래그 앤 드롭 방식으로 앱을 쉽게 개발할 수 있는 노코드 플랫폼. Google Sheet와 연동해 데이터를 활용할 수 있으며, 빠른 프로토타이핑이 가능함.

1) 디지털 전환의 정의

디지털 전환(Digital Transformation)은 기업이나 조직의 모든 영역에 디지털 기술을 통합하는 과정을 일컫습니다. 이는 기업의 프로세스, 제품, 운영 및 기술을 디지털화하여 지속적이고 신속한 고객 중심의 경영을 가능하게 합니다. 우버(Uber), 에어비앤비(AirBnB), 넷플릭스(Netflix)는 디지털 전환의 초기 사례로 모바일 및 클라우드 컴퓨팅 기술을 사용하여 기존 산업을 재편하였습니다. 최근의 디지털 전환은 인공지능과 자동화와 같은 기술로 워크플로우를 간소화하고 고객경험을 개인화하고, 의사결정을 개선하고 기업이 새로운 기회에 보다 빠르고 효과적으로 대응하고 있습니다.

디지털 전환의 영역을 크게 5가지로 나누면 비즈니스 및 운영 모델, 프로세스, 제품, 직원 경험, 고객 경험으로 나눌 수 있습니다. 그 중 두 가지를 예로 기술하겠습니다. 비즈니스 및 운영 모델 영역에서의 디지털 전환은 고객을 포함한 이해 관계자에게 제품이나 서비스를 제공하는 방식의 근본적인 변화(넷플릭스의 물리적 디스크에서 디지털 스트리밍으로의 변화)를 의미합니다. 또 직원 경험에서의 디지털 변환도 중요한데, 예로는 직원 간 협업 도구 제공, 재택/원격 근무 지원이 있습니다.

마지막으로 이런 디지털 전환을 위해서는 클라우드 컴퓨팅, 모바일 기술, IoT 기술, 인공지능과 기계학습, 자동화, 블록체인 등 디지털 기술이 뒷받침되어야 합니다.

#프레딕스 #GE #제너럴

2) GE사의 디지털 전환 시도와 실패에 관한 분석과 교훈

GE는 2013년 디지털 전환을 목표로 산업 인터넷용 소프트웨어 플랫폼 "Predix"를 제작하였습니다. 처음에는 사업이 순조로워 GE의 다양한 사업부에서 10억 달러 이상의 매출을 올렸지만, 결과적으로는 성공하지 못하고 GE는 소프트웨어 자회사인 GE 디지털을 매각하였습니다.

Predix의 실패에서 제일 큰 문제는 시장에 대한 이해를 충분히 하지 않고 사업을 진행한 것입니다. Predix는 제조업체들이 데이터를 실시간 분석하고 활용할 수 있도록 하려 했지만, 플랫폼을 판매하려고 했던 국가별로 산업 환경과 특성이 다른 것을 고려하지 않고 미국 기준으로만 플랫폼을 설계해 미국 외 시장으로의 진출에 한계가 있었습니다. 또한 경영진은 디지털 전환만 하면 막대한 수익을 거둘 것으로 예상하였지만, 실상은 그렇지 않았고 막대한 투자비용만 발생하였습니다. 또한 GE가 Predix를 설계할 때 한 플랫폼으로 모든 산업 현장에 적용을 목표로 하였지만, 실제로는 각 산업마다 요구 사항이 다르고 복잡하다는 특성이 있어서 판매량이 저조하였습니다.

따라서 이 GE의 실패로부터 우리는 디지털 전환을 할 때에는 특정 산업이나 시장에 대한 완전한 이해를 바탕으로

로 디지털 전환을 해야 한다는 것을 알 수 있습니다. 또한 Predix처럼 너무 광범위하게 디지털 전환을 시도하기 보다는 우선적으로는 소규모의 전환을 시도하는 것이 바람직하다고 생각합니다.

3) 성공한 디지털 전환의 사례 (스타벅스 제외)

디지털 전환의 사례로는 “모바일 페이(간편결제)”가 있습니다. 전통적인 금융/결제 산업에서는 현금결제나 물리적 카드(플라스틱 카드)가 주를 이루었지만, 최근 10년 내에 모바일 결제 시장이 커지고 있습니다. 국내의 대표적인 모바일 결제 서비스는 네이버페이, 카카오페이, PAYCO 등이 있으며, 국제적으로 유명한 서비스는 애플페이, 알리페이, 구글페이 등이 있습니다. 기사에 따르면, 전체 결제 비중에서 모바일 기기를 이용한 결제가 절반을 넘는다고 합니다. 우리 학교에서도 학생식당과 카페에서 PAYCO 서비스를 사용할 수 있는 만큼 모바일 페이는 성공한 디지털 전환의 사례라고 생각합니다.

#휴리스틱이 뭐야?

휴리스틱(Heuristic)은 문제 해결이나 의사 결정을 위해 사용하는 간편한 규칙이나 경험 기반의 방법을 의미해. 엄밀한 수학적 계산 없이도 빠르게 결론에 도달할 수 있도록 도와주는 전략인데, 보통 정확성보다는 속도와 효율성을 중시할 때 사용해.

휴리스틱은 복잡한 문제를 풀 때 최적의 답을 찾기 어렵거나 시간이 많이 걸릴 때, 적당히 만족스러운 해결책을 찾는 데 유용해. 예를 들어, 탐색 알고리즘에서 휴리스틱은 목표 상태에 얼마나 가까운지를 나타내는 추정치로 사용돼.

#폭포수 방식과 애자일 방식

폭포수 방식 (Waterfall)

- 순차적: 각 단계(요구사항 분석 → 설계 → 구현 → 테스트 → 배포)를 차례로 진행.
- 유연성 낮음: 중간에 요구사항을 변경하기 어려움.
- 장점: 명확한 구조와 철저한 문서화.
- 단점: 긴 개발 주기와 변경 어려움.

애자일 방식 (Agile)

- 반복적, 유연함: 짧은 주기로 소프트웨어를 개발하고, 피드백을 반영해 개선.
- 유연성 높음: 요구사항 변경에 빠르게 대응 가능.
- 장점: 빠른 피드백 반영, 사용자 만족도 높음.
- 단점: 문서화 부족, 요구사항 자주 변경 시 관리 어려움.

폭포수는 계획에 따라 진행되고, 애자일은 유연성과 피드백을 중시합니다.

#CCL

CCL(Creative Commons License)은 저작물에 대해 저작자가 설정한 이용 허락 조건을 명시하는 라이선스입니다. CCL을 사용하면 저작자가 자신의 저작물을 다른 사람들이 어떻게 사용할 수 있는지(복제, 배포, 수정 등)를 미리 정의할 수 있습니다.

CCL은 몇 가지 주요 조건을 조합하여 다양한 라이선스 형태로 제공됩니다:

- BY (저작자 표시): 저작자를 반드시 표시해야 함.
- NC (비영리): 비영리 목적으로만 사용 가능.
- ND (변경 금지): 저작물을 변경하거나 2차 저작물을 만들 수 없음.
- SA (동일 조건 변경 허락): 수정된 저작물도 동일한 라이선스로 배포해야 함.

이 조합에 따라 자유롭게 저작물을 이용할 수 있는지, 상업적 목적으로도 사용 가능한지 등이 결정됩니다.

#EULA 최종 사용자 사용권 계약

EULA(End-User License Agreement, 최종 사용자 사용권 계약)는 소프트웨어나 디지털 제품의 최종 사용자가 해당 소프트웨어를 사용할 수 있는 권리를 정의하는 법적 계약입니다. 사용자가 소프트웨어를 설치하거나 사용할 때 반드시 동의해야 하는 조건을 명시한 문서로, 보통 다음과 같은 내용을 포함합니다:

- 사용 허가 범위: 소프트웨어를 어떻게 사용할 수 있는지 (예: 개인 용도, 상업적 용도, 설치 가능한 기기의 수).
- 복제 및 배포 제한: 소프트웨어를 복제하거나 재배포하는 행위를 금지.
- 수정 금지: 소프트웨어의 소스 코드를 변경하거나 역공학(리버스 엔지니어링)하는 것을 금지.
- 책임 한계: 소프트웨어로 인해 발생하는 문제에 대해 개발자가 책임을 지지 않는다는 내용.

EULA는 사용자의 권리와 제한을 명확히 하고, 소프트웨어 개발자의 지적 재산을 보호하기 위한 중요한 법적 문서입니다.

#GNU #GPL

GNU GPL(GNU General Public License, GNU 일반 공중 사용 허가서)은 자유 소프트웨어 재단(Free Software Foundation, FSF)이 만든 오픈 소스 라이선스입니다. GNU GPL은 소프트웨어의 자유로운 사용, 수정, 배포를 허용하면서도 소프트웨어의 자유를 유지하기 위한 강력한 조건을 포함합니다.

○ 주요 특징

1. 자유 소프트웨어: 소프트웨어를 누구나 자유롭게 사용, 수정, 배포할 수 있도록 허용합니다.
2. 소스 코드 공개 의무: GPL로 배포된 소프트웨어를 수정하거나 재배포할 경우, 수정된 소프트웨어의 소스 코드를 공개해야 합니다.
3. 동일한 라이선스 유지 (Copyleft): GPL 소프트웨어를 수정하거나 그 소스 코드를 이용해 새로운 소프트웨어를 만들 경우, 새로운 소프트웨어도 반드시 GPL로 배포해야 합니다. 즉, 파생된 작업은 동일한 조건으로 공유되어야 합니다.
4. 상업적 사용 가능: GPL 소프트웨어는 상업적으로도 사용할 수 있지만, 배포할 경우 소스 코드를 반드시 공개해야 하며, 파생 제품도 GPL로 제공해야 합니다.

○ 목적

GNU GPL은 소프트웨어의 자유를 보장하는 동시에, 그 자유가 다른 사람들이 사용하는 과정에서 제한되지 않도록 합니다. 이 라이선스는 사용자와 개발자가 소프트웨어의 권한을 공유하면서도, 그 자유를 보호하기 위한 강력한 도구입니다.

○ 예시

리눅스(Linux) 커널을 포함한 많은 유명한 오픈 소스 소프트웨어들이 GNU GPL을 따르고 있습니다.

#오픈소스

오픈 소스 소프트웨어(OS)는 무료로 제공되는 소스 코드를 누구나 사용, 수정, 배포할 수 있도록 허용하는 소프트웨어입니다. 하지만 무료로 제공된다고 해서 오픈 소스를 기반으로 한 수익 모델이 없다는 것은 아닙니다. 오픈 소스 소프트웨어의 수익 모델은 다양한 방식으로 운영될 수 있으며, 대표적인 오픈 소스 사업 모델은 다음과 같습니다.

1. 서비스 및 지원

- 비즈니스 모델: 소프트웨어 자체는 무료로 제공되지만, 설치, 설정, 유지보수, 기술 지원과 같은 서비스를 유료로 제공합니다.
- 예시: Red Hat은 Linux 배포판을 무료로 제공하지만, 기업 고객에게는 기술 지원과 업그레이드 서비스를 제공하여 수익을 창출합니다.

2. 호스팅 및 클라우드 서비스

- 비즈니스 모델: 오픈 소스 소프트웨어를 클라우드 플랫폼에서 제공하고, 이를 호스팅하거나 관리하는 서비스를 유료로 운영합니다.
- 예시: GitLab이나 Elasticsearch는 무료 오픈 소스 버전을 제공하지만, 클라우드 기반의 호스팅 및 관리형 서비스를 제공하여 수익을 얻습니다.

3. 이중 라이선스 (Dual Licensing)

- 비즈니스 모델: 동일한 소프트웨어에 대해 오픈 소스 라이선스와 상업적 라이선스 두 가지를 제공하여, 상업적으로 사용하려는 기업에게 유료 라이선스를 판매합니다. 상업적 라이선스는 GPL과 같은 제한을 피할 수 있습니다.

- 예시: MySQL은 오픈 소스 버전을 제공하면서도, 상업적인 사용을 위해 별도의 유료 라이선스를 판매합니다.

4. 프리미엄 모델 (Freemium)

- 비즈니스 모델: 기본적인 기능은 무료로 제공하고, 고급 기능이나 추가 도구는 유료로 제공하는 방식입니다.

- 예시: JetBrains의 IntelliJ IDEA는 오픈 소스 버전(Community Edition)을 제공하면서, 고급 기능을 포함한 유료 버전(Ultimate Edition)을 별도로 판매합니다.

5. 컨설팅 및 맞춤형 개발

- 비즈니스 모델: 오픈 소스 소프트웨어를 기반으로 특정 기업의 요구에 맞는 맞춤형 개발을 제공하고, 이에 대한 컨설팅 서비스를 제공합니다.

- 예시: 많은 기업들이 오픈 소스 솔루션을 자신의 필요에 맞게 최적화하고, 이를 위해 전문가 컨설팅 서비스를 받습니다.

6. 기부 및 후원

- 비즈니스 모델: 소프트웨어를 무료로 제공하되, 기부나 후원을 통해 수익을 창출합니다. 이 모델은 주로 오픈 소스 커뮤니티나 비영리 단체에서 많이 사용됩니다.

- 예시: 오픈 소스 프로젝트인 Blender는 사용자와 기업의 후원으로 운영됩니다.

7. 애드온 및 플러그인 판매

- 비즈니스 모델: 기본 소프트웨어는 무료로 제공하지만, 추가적인 기능을 제공하는 애드온이나 플러그인을 유료로 판매합니다.

- 예시: WordPress는 기본 CMS를 무료로 제공하지만, 유료 테마나 플러그인을 통해 수익을 창출합니다.

오픈 소스 사업 모델의 특징

- 소프트웨어의 자유성: 사용자는 소프트웨어를 자유롭게 사용할 수 있지만, 부가 서비스나 고급 기능을 통해 수익을 창출.

- 커뮤니티 기여: 많은 오픈 소스 프로젝트는 커뮤니티의 기여와 참여로 성장하며, 이를 기반으로 사업 모델을 구축함.

- 유연성: 다양한 수익 모델을 결합하여 운영할 수 있으며, 소프트웨어 배포와 관련된 비용은 줄이면서 다른 방식으로 수익을 창출할 수 있음.

오픈 소스 소프트웨어는 단순히 무료로 제공되는 소프트웨어를 넘어, 이를 바탕으로 한 지속 가능한 비즈니스 모델을 형성할 수 있는 가능성을 제공합니다.

#크리티컬 시스템

크리티컬 시스템(Critical System)은 오류나 실패가 발생할 경우 사람의 생명, 재산, 환경, 또는 시스템 전체에 심각한 피해를 입힐 수 있는 시스템을 말합니다. 이러한 시스템은 높은 신뢰성과 안전성을 요구하며, 실패 시 큰 위험을 초래할 수 있기 때문에 엄격한 설계, 테스트, 검증 과정을 거쳐야 합니다.

디지털 워터마킹(Digital Watermarking)과 디지털 핑거프린팅(Digital Fingerprinting)은 모두 디지털 콘텐츠를 보호하는 기술이지만, 그 목적과 방식에서 차이가 있습니다.

1. 디지털 워터마킹 (Digital Watermarking)

- 목적: 워터마킹은 디지털 콘텐츠(이미지, 동영상, 오디오 등)에 보이지 않는 표식을 삽입해 저작권 소유권을 주장하거나 출처를 밝히기 위한 기술입니다. 이 표식은 일반적으로 콘텐츠의 무결성을 유지하면서 눈에 보이지 않게 삽입됩니다.

- 용도: 저작권 보호, 위조 방지, 출처 확인 등에 사용됩니다.
- 특징:
 - 콘텐츠 자체에 영구적으로 삽입되며, 제거가 어렵습니다.
 - 일반적으로 동일한 워터마크가 모든 콘텐츠에 삽입됩니다(예: 저작권자의 이름이나 로고).
 - 공격에 강해야 하고, 품질에 영향을 미치지 않아야 함.

2. 디지털 핑거프린팅 (Digital Fingerprinting)

- 목적: 핑거프린팅은 각 사용자에게 고유한 식별자를 삽입하여 특정 사용자가 어떤 콘텐츠를 배포하거나 공유했는지 추적하기 위한 기술입니다.
- 용도: 불법 복제 및 배포 추적, 콘텐츠 유출 방지.
- 특징:
 - 각 사용자마다 고유한 핑거프린트를 삽입하여 그들의 행위를 추적할 수 있습니다.
 - 핑거프린트는 누가 콘텐츠를 불법적으로 배포했는지 식별하는 데 사용됩니다.
 - 주로 저작권이 있는 콘텐츠의 불법 복제나 배포를 방지하는 데 쓰입니다.

차이점 요약

- 목적: 워터마킹은 콘텐츠의 소유권을 명시하고, 핑거프린팅은 개별 사용자를 추적합니다.
- 삽입 방식: 워터마킹은 모든 사용자에게 동일한 정보를 삽입하지만, 핑거프린팅은 각 사용자마다 다르게 삽입됩니다.
- 용도: 워터마킹은 주로 저작권 표시에 사용되며, 핑거프린팅은 불법 배포 추적에 사용됩니다.

이 두 기술은 디지털 콘텐츠 보호의 중요한 부분으로, 각각의 목적에 맞게 활용됩니다.

#파이프라이닝

프로세서의 파이프라이닝은 마치 공장의 조립 라인처럼, 하나의 명령어를 여러 작은 단계로 나누어 동시에 여러 명령어를 처리하는 방식입니다. 이를 통해 단위 시간당 처리되는 명령어의 수를 늘려 프로세서의 성능을 향상시키는 것이 목표입니다.

왜 파이프라이닝을 사용할까요?

성능 향상: 하나의 명령어가 완전히 처리되기를 기다리지 않고, 여러 명령어를 동시에 처리하는 단계별로 나누어 처리하기 때문에 전체적인 처리 속도가 빨라집니다.

효율성 증대: 프로세서의 각 부분을 더 효율적으로 활용할 수 있게 됩니다. 예를 들어, 한 명령어가 메모리에서 데이터를 가져오는 동안 다른 명령어는 연산을 수행하는 식으로 말이죠.

#GPU

GPU는 그래픽 처리 장치의 약자로, 컴퓨터에서 이미지나 영상을 빠르게 처리하기 위해 설계된 전자 회로입니다. CPU와 비교하면,

CPU는 다양한 작업을 순차적으로 처리하는 데 뛰어납니다.

GPU는 동일한 작업을 매우 많은 양으로 동시에 처리하는 데 특화되어 있습니다.

GPU가 필요한 이유

고해상도 게임: 복잡한 그래픽을 실시간으로 처리하여 부드러운 화면을 제공합니다.

3D 모델링: 3차원 모델을 생성하고 편집하는 작업을 가속화합니다.

영상 편집: 고화질 영상을 빠르게 편집하고 효과를 적용합니다.

딥러닝: 인공지능 학습에 필요한 방대한 데이터를 빠르게 처리합니다.

간단히 말해서, GPU는 컴퓨터 그래픽을 빠르게 처리하여 우리가 더욱 생생하고 현실적인 디지털 경험을 할 수 있도록 돕는 역할을 합니다.

더 궁금한 점이 있다면 언제든지 물어보세요!

CPU와 GPU의 차이점이 더 궁금하다면: CPU와 GPU는 서로 다른 역할을 수행하며, 장단점이 있습니다.

GPU가 사용되는 분야가 더 궁금하다면: 게임, 영상 편집, 딥러닝 외에도 다양한 분야에서 GPU가 활용되고 있습니다.

#범용 기계

범용 기계는 특정한 한 가지 작업만을 수행하도록 설계된 특수 목적 기계와 달리, 다양한 작업을 수행할 수 있도록 설계된 기계를 의미합니다. 즉, 범용 기계는 활용도가 높고 유연성이 뛰어나 여러 분야에서 다양하게 사용될 수 있습니다.

#N문제 NP문제

P 문제와 NP 문제는 컴퓨터 과학에서 매우 중요하고 오랫동안 연구되어 온 개념입니다. 이 두 문제는 문제의 복잡도를 기준으로 분류하며, 특히 P vs. NP 문제는 밀레니엄 문제 중 하나로 꼽힐 만큼 풀리지 않은 난제입니다.

P 문제란?

다항 시간(Polynomial Time) 안에 해결 가능한 문제를 의미합니다.

즉, 문제의 크기가 커질수록 해결하는 데 걸리는 시간이 다항식적으로 증가하는 문제입니다.

예를 들어, 정렬 문제는 문제의 크기(데이터의 개수)에 따라 시간 복잡도가 $n \log n$ 정도로 증가하므로 P 문제에 속합니다.

NP 문제란?

정답이 주어졌을 때 다항 시간 안에 정답의 정확성을 검증할 수 있는 문제를 의미합니다.

하지만, 정답을 찾는 데는 다항 시간이 걸리지 않을 수도 있습니다.

예를 들어, 여행하는 외판원 문제는 모든 도시를 한 번씩 방문하고 다시 출발점으로 돌아오는 최단 경로를 찾는 문제인데, 모든 가능한 경로를 다 확인해야 하므로 다항 시간 안에 해결하기 어렵습니다. 하지만 어떤 경로가 주어지면 이 경로가 최단 경로인지 아닌지는 쉽게 확인할 수 있습니다.

P vs. NP 문제

P 문제는 NP 문제의 부분집합입니다. 즉, 모든 P 문제는 NP 문제이기도 합니다.

모든 NP 문제가 P 문제인지 아닌지는 아직 증명되지 않은 문제입니다.

만약 모든 NP 문제가 P 문제라면, 세상의 많은 어려운 문제들이 쉽게 해결될 수 있지만, 현재까지는 그러한 알고리즘이 발견되지 않았습니다.

왜 P vs. NP 문제가 중요할까요?

암호학: 많은 암호 시스템은 NP 문제를 기반으로 합니다. 만약 모든 NP 문제가 P 문제라면, 현재 사용되는 많은 암호 시스템이 쉽게 해킹될 수 있습니다.

알고리즘 설계: 어떤 문제를 해결하기 위해 어떤 알고리즘을 사용해야 할지 결정하는 데 중요한 기준이 됩니다.

컴퓨터 과학의 기본적인 질문: 컴퓨터가 해결할 수 있는 문제의 한계를 탐구하는 데 중요한 의미를 가집니다.

결론

P 문제와 NP 문제는 컴퓨터 과학에서 매우 중요한 개념이며, P vs. NP 문제는 아직 풀리지 않은 난제입니다. 이 문제의 해결은 컴퓨터 과학뿐만 아니라 인류의 미래에도 큰 영향을 미칠 수 있습니다.

더 궁금한 점이 있다면 언제든지 질문해주세요.

NP-완전 문제: NP 문제 중에서 가장 어려운 문제에 대해 알고 싶으시다면?

NP-난해 문제: NP-완전 문제와 비슷하지만 약간 다른 개념에 대해 알고 싶으시다면?

P vs. NP 문제의 역사: 이 문제가 어떻게 시작되었고, 어떤 연구들이 진행되었는지 알고 싶으시다면?

다음과 같은 질문을 해보세요:

"NP-완전 문제와 NP-난해 문제의 차이점은 무엇인가요?"

"P vs. NP 문제가 풀리면 어떤 일이 일어날까요?"

"현재 P vs. NP 문제를 해결하기 위한 연구는 어떤 방향으로 진행되고 있나요?"

휴리스틱 알고리즘은 최적의 해를 보장하지는 않지만, 빠르게 근사적인 해를 찾는 데 효과적인 알고리즘입니다. 특히, NP-완전 문제처럼 정확한 해를 찾기 어려운 문제를 해결할 때 많이 사용됩니다.

휴리스틱 알고리즘의 특징

빠른 실행 속도: 최적의 해를 찾기 위해 모든 경우의 수를 따지는 대신, 경험적 지식이나 추측을 바탕으로 빠르게 해를 찾습니다.

근사적인 해: 항상 최적의 해를 보장하는 것은 아니지만, 실제 문제에 적용했을 때 충분히 좋은 결과를 내는 경우가 많습니다.

문제에 특화된 규칙: 각 문제에 맞는 특별한 규칙이나 전략을 사용하여 해를 찾습니다.

왜 휴리스틱 알고리즘을 사용할까요?

NP-완전 문제 해결: NP-완전 문제는 모든 경우의 수를 따져야 하기 때문에 시간이 오래 걸립니다. 휴리스틱 알고리즘을 사용하면 빠르게 근사적인 해를 찾을 수 있습니다.

실시간 처리: 실시간으로 문제를 해결해야 하는 경우, 최적의 해를 찾기 위해 너무 많은 시간을 소비할 수 없습니다.

휴리스틱 알고리즘은 빠른 응답 시간을 제공합니다.

복잡한 문제 단순화: 복잡한 문제를 간단한 규칙으로 해결하여 문제 해결 과정을 단순화할 수 있습니다.

휴리스틱 알고리즘의 예시

A 탐색 알고리즘: 최단 경로 문제를 해결하는 알고리즘으로, 목표 지점까지의 예상 거리를 계산하여 유망한 경로를 먼저 탐색합니다.

유전 알고리즘: 생물의 진화 과정을 모방하여 최적의 해를 찾는 알고리즘입니다.

최근접 이웃 알고리즘: 데이터 클러스터링 문제를 해결하는 알고리즘으로, 가장 가까운 데이터끼리 묶어 클러스터를 형성합니다.

#접근 제어 vs #사용 제어

DRM에서 접근 제어와 사용 제어의 차이점

DRM(Digital Rights Management)은 디지털 콘텐츠의 저작권을 보호하고 불법 복제를 방지하기 위한 기술입니다. 이러한 DRM 시스템에서 접근 제어와 사용 제어는 콘텐츠 보호를 위한 두 가지 중요한 기능입니다.

접근 제어 (Access Control)

정의: 누가 특정 콘텐츠에 접근할 수 있는지를 결정하는 기능입니다.

목적:

인증: 사용자의 신원을 확인하여 허가된 사용자만 접근할 수 있도록 합니다.

권한 부여: 사용자에게 부여된 권한에 따라 접근 가능한 기능이나 콘텐츠를 제한합니다.

방법:

아이디/비밀번호: 가장 기본적인 인증 방식입니다.

생체 인식: 지문, 얼굴 인식 등을 통해 사용자를 식별합니다.

디지털 서명: 콘텐츠의 무결성을 보장하고 사용자의 신원을 확인합니다.

예시:

특정 웹사이트에 로그인해야만 콘텐츠를 볼 수 있도록 하는 것

특정 기기에서만 콘텐츠를 재생할 수 있도록 제한하는 것

사용 제어 (Usage Control)

정의: 콘텐츠를 어떻게 사용할 수 있는지를 제한하는 기능입니다.

목적:

복제 방지: 콘텐츠의 불법 복제를 막습니다.

사용 환경 제한: 특정 환경에서만 콘텐츠를 사용할 수 있도록 제한합니다.

사용 횟수 제한: 콘텐츠 사용 횟수를 제한합니다.

방법:
워터마크: 콘텐츠에 숨겨진 정보를 삽입하여 불법 복제 여부를 추적합니다.
암호화: 콘텐츠를 암호화하여 무단 접근을 차단합니다.
DRM 칩: 하드웨어 기반의 보안 칩을 사용하여 콘텐츠를 보호합니다.
예시:
eBook을 한 번에 한 기기에서만 볼 수 있도록 제한하는 것
영화를 다운로드하여 오프라인에서 시청할 수 있는 횟수를 제한하는 것
두 가지의 차이점 요약

구분	접근 제어	사용 제어
대상	누가 접근할 수 있는지	어떻게 사용할 수 있는지
목적	인증, 권한 부여	복제 방지, 사용 환경 제한
방법	아이디/비밀번호, 생체 인식	디지털 서명, 워터마크, 암호화, DRM 칩

결론
접근 제어와 사용 제어는 서로 보완적인 관계를 가지며, DRM 시스템에서 콘텐츠를 안전하게 보호하는 데 중요한 역할을 합니다. 접근 제어는 누가 콘텐츠에 접근할 수 있는지를 결정하고, 사용 제어는 콘텐츠를 어떻게 사용할 수 있는지를 제한함으로써 저작권 침해를 방지하고 콘텐츠의 가치를 보호합니다.
예를 들어, 유료 동영상 서비스의 경우, 아이디와 비밀번호를 통해 인증된 사용자만 접근할 수 있도록 접근 제어를 하고, 다운로드하여 오프라인에서 시청할 수 있는 횟수를 제한하거나, 특정 기기에서만 재생할 수 있도록 사용 제어를 합니다.

클린 룸 개발(Clean Room Development)

****지적 재산을 침해하지 않고 소프트웨어나 하드웨어를 개발****하는 방법론을 말합니다. 이 방식은 주로 경쟁 제품의 특허나 저작권을 침해할 위험이 있는 경우에 사용됩니다. 클린 룸 개발은 두 개의 독립적인 팀이 협력하여 이루어지며, 각 팀은 다음과 같은 역할을 합니다:

- **분석 팀(또는 규격 팀)**:** 기존 제품을 분석하여 해당 소프트웨어나 하드웨어의 기능을 문서화하지만, 실제 코드를 보거나 이를 직접 참조하지는 않습니다. 이 팀은 기본적으로 어떤 기능이 필요한지를 정의하는 역할을 맡습니다.
- **개발 팀**:** 분석 팀이 제공한 문서를 기반으로 독립적으로 새로운 소프트웨어 또는 하드웨어를 개발합니다. 이 팀은 기존 제품의 소스 코드를 보거나 사용하는 것이 금지됩니다.

이 방식을 사용하면, 지적 재산을 침해하지 않고도 비슷한 기능을 제공하는 새로운 제품을 개발할 수 있습니다. 클린 룸 개발은 법적으로 안전하게 경쟁사 제품과 유사한 기능을 만들 때 자주 사용되며, 특히 소프트웨어의 경우 리버스 엔지니어링과는 다른 접근 방식입니다.

이 방법은 1980년대 IBM PC 호환 컴퓨터를 개발할 때 널리 사용된 바 있으며, 오늘날에도 특정 기술을 우회하려는 경우에 많이 사용됩니다.

#오라클(Oracle)과 구글(Google)

간의 API 저작권 분쟁은 주로 **Java API**의 사용에 관한 것으로, 이 사건은 소프트웨어 개발에서 API 사용에 대한 저작권 및 공정사용(fair use)의 경계를 다룬 중요한 판례로 꼽힙니다.

사건 배경

오라클은 2010년에 구글을 상대로 소송을 제기했습니다. 구글이 **Android 운영체제**를 개발하는 과정에서 **Java API**의 일부를 무단으로 사용했다고 주장했죠. Java는 원래 Sun Microsystems의 소프트웨어였으나, 2010년에 오라클이 Sun을 인수하면서 해당 소프트웨어의 권리를 가지게 되었습니다.

구글은 Java의 37개의 API 패키지를 Android 개발에 사용했는데, 여기서 논란이 된 부분은 **API 설계**와 관련된 코드였습니다. 구글은 오라클의 Java API 코드를 직접 복사한 것이 아니라, 해당 API의 함수 이름과 구조를 재사용했다고 주장했습니다.

핵심 쟁점

1. **API에 저작권이 있는가?**

API 자체가 저작권으로 보호될 수 있는지 여부는 이 사건의 핵심 쟁점 중 하나였습니다. 오라클은 API도 저작권 법으로 보호된다고 주장한 반면, 구글은 API는 일종의 "기능적 요소"로서 창작적 요소보다는 기술적 규약이므로 저작권 보호 대상이 아니라고 주장했습니다.

2. **공정 사용(Fair Use) 여부**

구글은 설령 API가 저작권 보호 대상이라고 하더라도, 자신들의 사용은 공정 사용에 해당한다고 주장했습니다. 공정 사용은 저작권이 있는 콘텐츠를 제한적으로 사용해도 되는 경우를 말하며, 주로 학술적, 비영리적, 교육적 목적으로 허용됩니다.

판결 결과

이 사건은 여러 단계의 법적 과정을 거쳤고, 결국 미국 연방대법원까지 가게 되었습니다.

- **2021년 4월 5일**: 미국 연방대법원은 **구글의 Java API 사용이 공정 사용(fair use)에 해당한다**는 판결을 내렸습니다. 대법원은 구글이 API를 사용한 목적이 Android 플랫폼에서 새로운 혁신적 제품을 만드는 것이었으며, 구글의 Java API 사용은 기존 창작물을 그대로 복제한 것이 아니라 새로운 맥락에서 변형해 사용한 것이라고 보았습니다. 따라서 구글의 사용은 공정 사용으로 인정되며 저작권을 침해하지 않았다는 결론이 내려졌습니다.

결론

구글은 오라클의 Java API를 불법적으로 사용하지 않았다는 판결을 받았으며, 대법원은 **공정 사용** 원칙에 따라 구글의 Java API 사용이 허용된다고 판단했습니다. 이는 소프트웨어 개발에서 API의 사용과 저작권 문제에 대해 중요한 법적 선례가 되었습니다.